

Manual de Utilização do PSE-Image

Manual ilustrado com imagens extraídas do vídeo de demonstração

Ambiente visual para montar, executar e avaliar pipelines de processamento de imagens em escala de cinza.

Projeto
PSE-Image

Documento
Manual ilustrado

Versão
Sem seções removidas

Sumário

- [1. Visão Geral](#)
 - [2. Requisitos](#)
 - [3. Estrutura do Projeto](#)
 - [4. Como Rodar](#)
 - [5. Como Usar a Interface](#)
 - [6. Blocos Disponíveis](#)
 - [7. Exemplos de Fluxos](#)
 - [8. Observações Técnicas](#)
 - [9. Encerramento](#)
-

1. Visão Geral

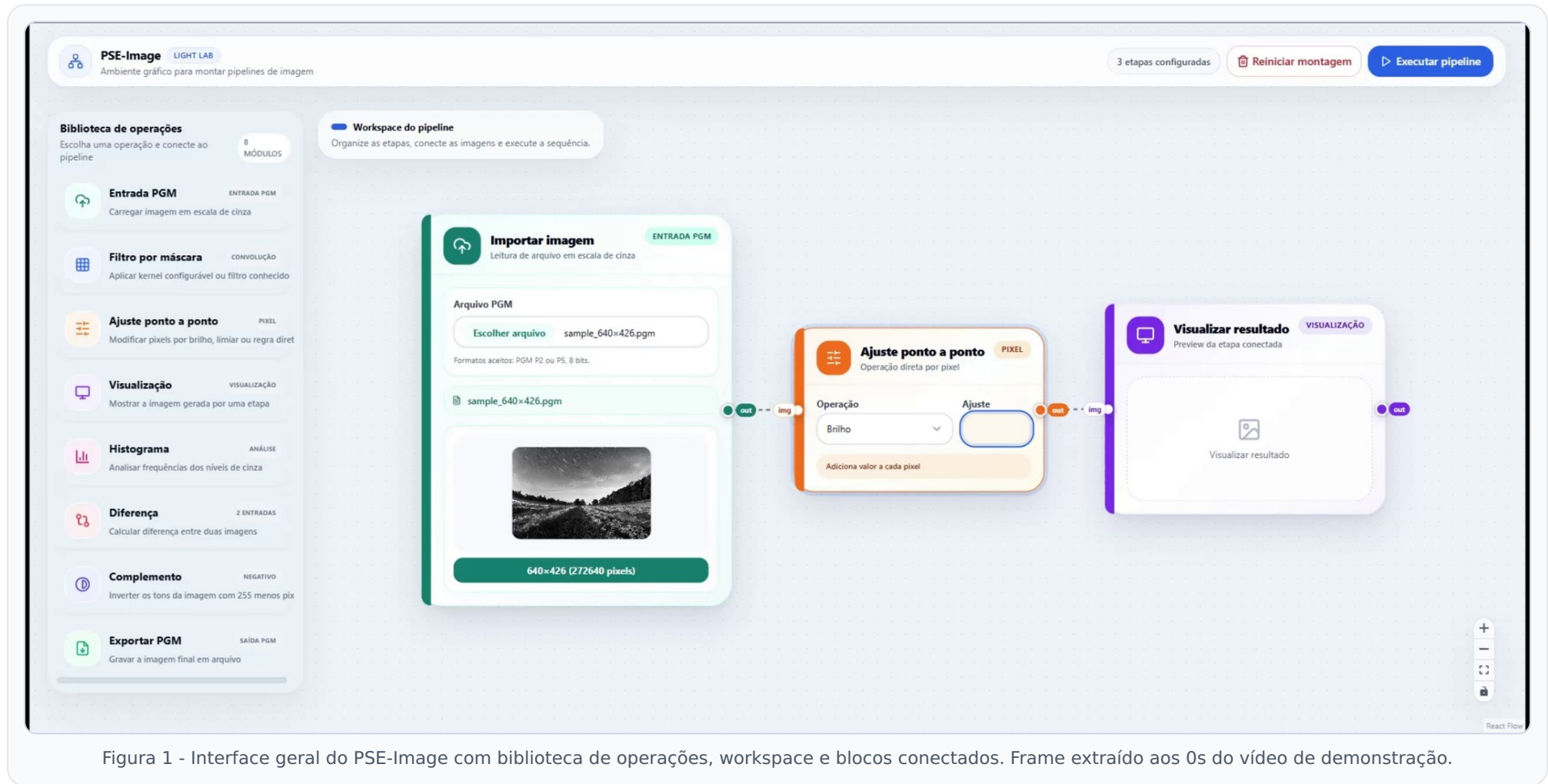


Figura 1 - Interface geral do PSE-Image com biblioteca de operações, workspace e blocos conectados. Frame extraído aos 0s do vídeo de demonstração.

O **PSE-Image** é um Problem-Solving Environment para processamento de imagens. Ele permite que o usuário monte fluxos de processamento por meio de blocos gráficos conectáveis, sem precisar escrever código.

O usuário pode:

- Carregar imagens PGM em escala de cinza.

- Conectar blocos para formar pipelines.
- Parametrizar operações de processamento.
- Executar o fluxo completo.
- Visualizar resultados intermediários ou finais.
- Analisar histograma.
- Salvar imagens processadas em PGM.

Arquitetura Geral

Camada	Tecnologia	Função
Frontend	React, TypeScript, Vite, Tailwind, React Flow	Interface gráfica com blocos conectáveis
Backend	Python, FastAPI, Pydantic	Upload de PGM e processamento do grafo
Formato de imagem	PGM <code>P2</code> e <code>P5</code>	Imagens acromáticas em 8 bits/pixel

2. Requisitos

2.1 Requisitos de Software

Instale os seguintes itens antes de executar o projeto:

Software	Versão recomendada	Uso
Node.js	18 ou superior	Executar o frontend
npm	Instalado com Node.js	Instalar dependências do frontend
Python	3.10 ou superior	Executar o backend
Navegador	Chrome, Edge ou Firefox	Acessar a interface

2.2 Requisitos das Imagens

O sistema aceita apenas imagens PGM em escala de cinza.

Item	Exigência
Extensão	.pgm
Formatos aceitos	P2 ASCII e P5 binário
Valor máximo	255
Tipo	Acromática, escala de cinza
Profundidade	8 bits por pixel

Arquivos .jpg, .png, .bmp ou outros formatos não são aceitos diretamente.

3. Estrutura do Projeto

```
pse-image-pai/  
  backend/  
    main.py  
    models.py  
    processor.py  
    requirements.txt  
    operations/  
      common.py  
      raw_reader/  
      point_operation/  
      convolution/  
      display/  
      histogram/  
      difference/  
      complement/  
      save/  
    output/  
  frontend/  
    src/  
    package.json  
    vite.config.ts  
  images_database/  
  README.md  
  MANUAL.md  
  TP_PA1.pdf
```

Pastas importantes

Caminho	Conteúdo
backend/operations/	Funcionalidades do backend, separadas por diretório
backend/output/	Arquivos PGM gerados pelo bloco de salvamento
frontend/src/	Código da interface gráfica
images_database/	Imagens PGM de exemplo/base de dados

4. Como Rodar

O backend e o frontend devem ficar abertos ao mesmo tempo, em terminais separados.

4.1 Backend

Abra um terminal na raiz do projeto.

Entre na pasta do backend:

```
cd backend
```

Crie o ambiente virtual:

```
python -m venv venv
```

Ative no Windows:

```
venv\Scripts\activate
```

Ative no Linux/macOS:

```
source venv/bin/activate
```

Instale as dependências:

```
pip install -r requirements.txt
```

Inicie a API:

```
python main.py
```

Backend disponível em:

```
http://localhost:8000
```

Rotas principais da API

Método	Rota	Descrição
GET	/	Informações básicas da API
GET	/health	Verifica se a API está ativa
POST	/upload-raw	Recebe <code>.pgm</code> e devolve pixels
POST	/process	Processa o grafo enviado pelo frontend

4.2 Frontend

Abra outro terminal na raiz do projeto.

Entre na pasta do frontend:

```
cd frontend
```

Instale as dependências:

```
npm install
```

Inicie a interface:

```
npm run dev
```

Frontend disponível em:

```
http://localhost:5173
```

Abra esse endereço no navegador.

5. Como Usar a Interface

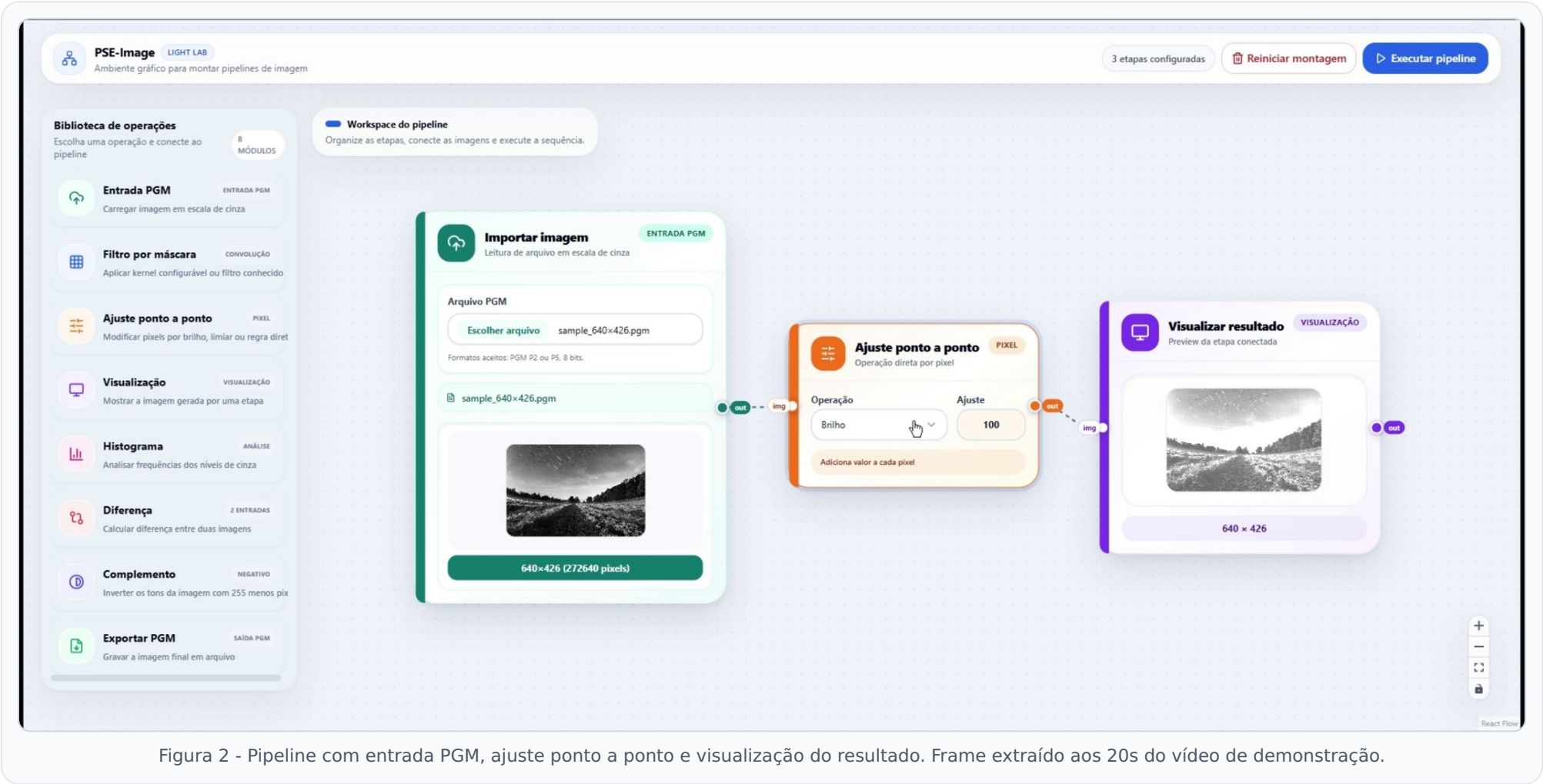


Figura 2 - Pipeline com entrada PGM, ajuste ponto a ponto e visualização do resultado. Frame extraído aos 20s do vídeo de demonstração.

5.1 Tela Principal

Ao abrir o sistema, você verá:

Área	Função
Barra superior	Mostra o nome do sistema e ações principais
Biblioteca de operações	Lista os blocos disponíveis
Workspace	Área central onde os blocos são adicionados e conectados

5.2 Adicionar um Bloco

Na biblioteca de operações, clique no bloco desejado.

Blocos disponíveis:

Bloco	Uso
Entrada PGM	Carregar imagem
Filtro por máscara	Aplicar convolução/filtros
Ajuste ponto a ponto	Brilho ou limiarização
Visualização	Exibir imagem
Histograma	Plotar frequências de cinza
Diferença	Comparar duas imagens
Complemento	Gerar negativo
Exportar PGM	Salvar imagem

5.3 Conectar Blocos

Cada bloco possui pontos de conexão:

Ponto	Local comum	Função
Entrada	Lado esquerdo	Recebe imagem ou dados
Saída	Lado direito	Envia resultado para outro bloco

Passos:

1. Clique e segure no ponto de saída de um bloco.
2. Arraste até o ponto de entrada de outro bloco.
3. Solte para criar a conexão.

O backend executa os blocos respeitando a ordem das conexões.

5.4 Processar o Fluxo

Depois de montar o pipeline:

1. Confira se a imagem foi carregada no bloco de entrada.
2. Confira se os blocos estão conectados corretamente.
3. Clique em `Executar pipeline`.

O frontend envia o grafo para o backend, o backend processa as etapas e devolve os resultados para os blocos.

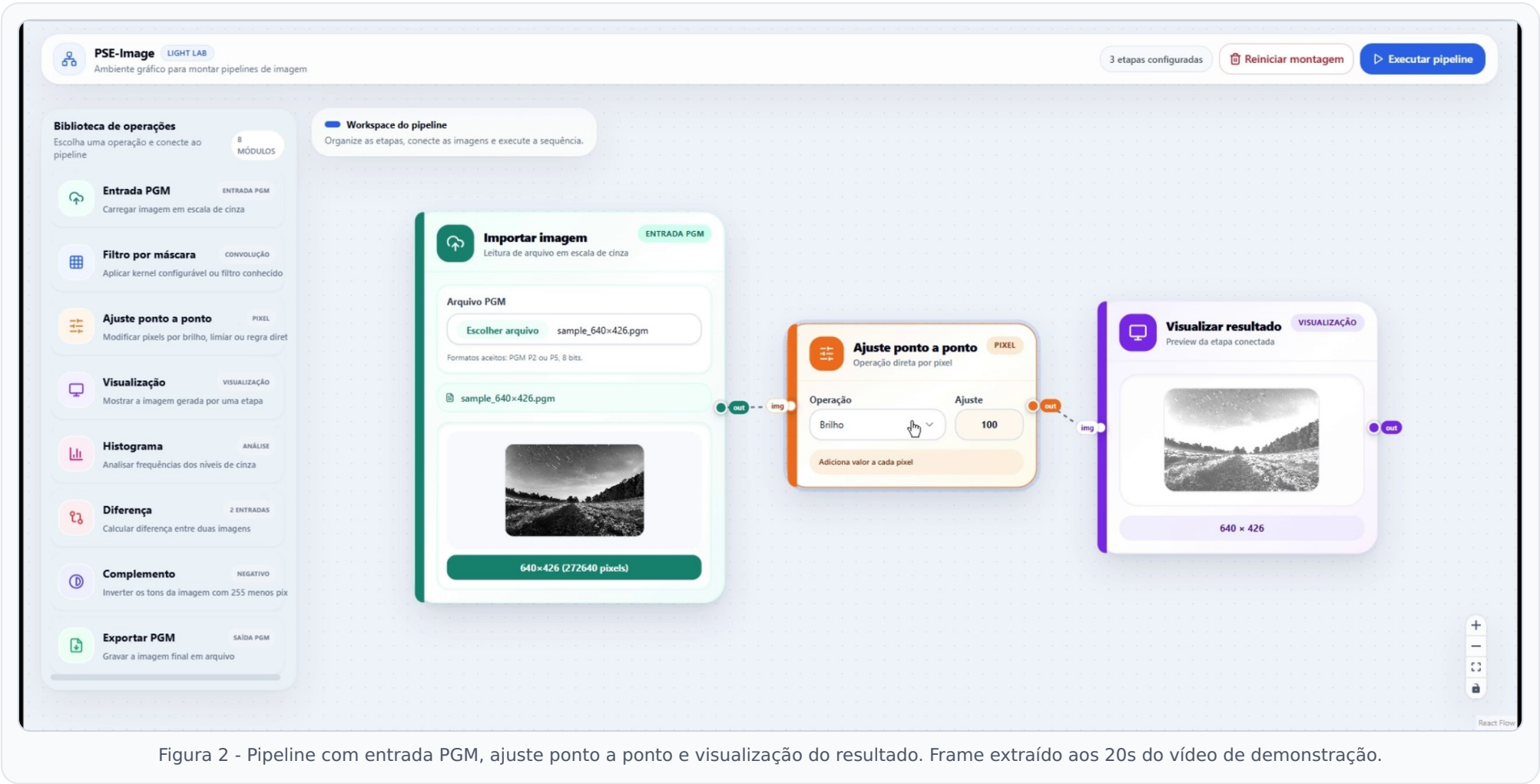
5.5 Limpar o Workspace

Para remover todos os blocos e conexões, clique em `Reiniciar montagem`.

O sistema pede confirmação antes de limpar o workspace.

6. Blocos Disponíveis

6.1 Entrada PGM



Função: carregar uma imagem PGM para iniciar o fluxo.

Como usar:

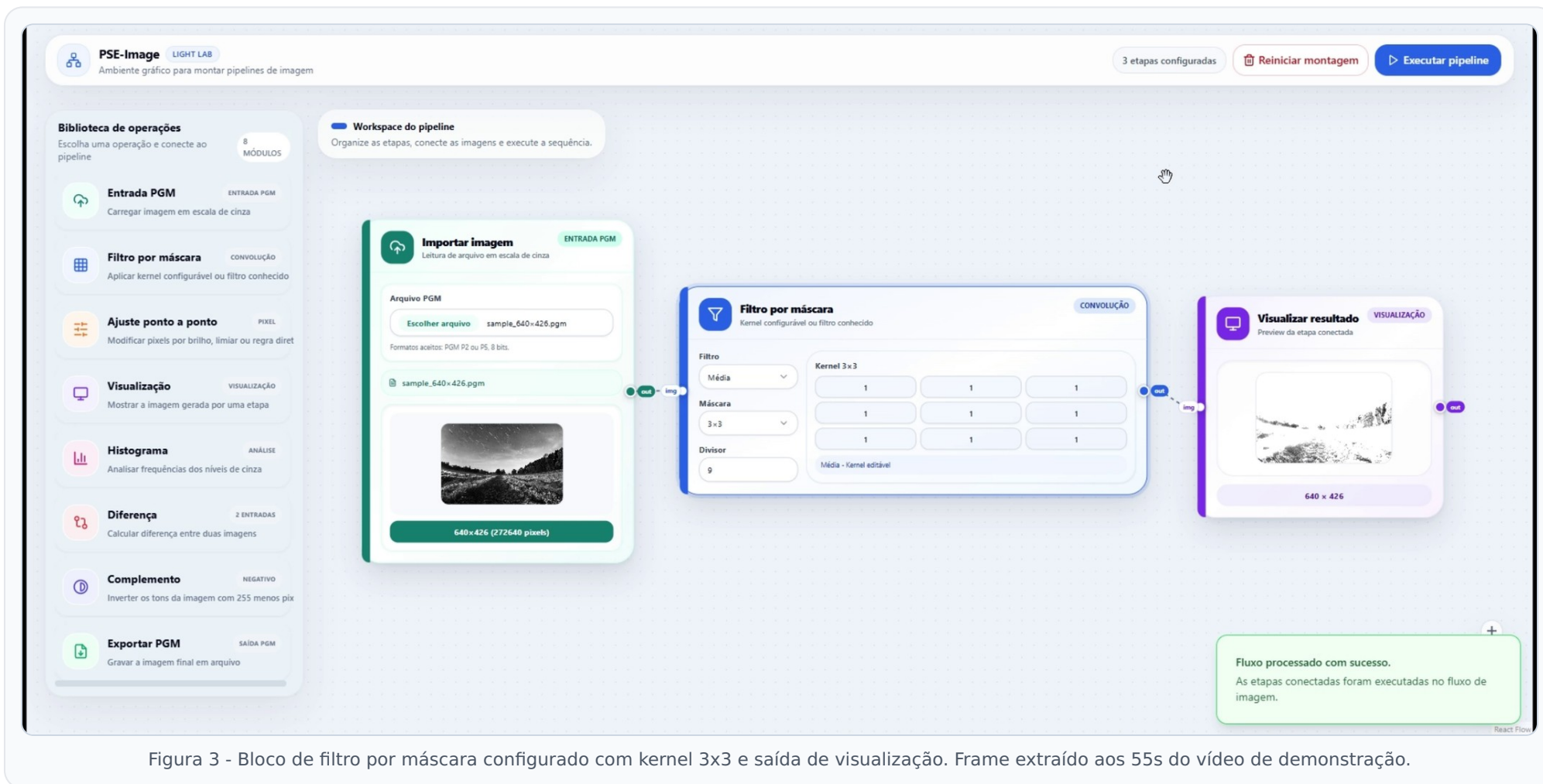
1. Adicione o bloco `Entrada PGM`.
2. Clique na área de upload.
3. Selecione um arquivo `.pgm`.
4. Aguarde o carregamento.

Validações feitas pelo backend:

Validação	Descrição
Extensão	O arquivo deve ser <code>.pgm</code>
Formato	Deve ser <code>P2</code> ou <code>P5</code>
Valor máximo	Deve ser <code>255</code>
Dimensões	Largura e altura precisam ser válidas
Pixels	A quantidade de pixels deve bater com as dimensões

Saída: imagem em escala de cinza no formato interno do sistema.

6.2 Filtro por Máscara



Função: aplicar convolução ou filtros locais sobre a imagem.

Filtros disponíveis:

Filtro	Efeito
Média	Suaviza a imagem
Mediana	Reduz ruído preservando bordas
Laplaciano	Destaca mudanças bruscas de intensidade
Kernel configurável	Permite pesos definidos pelo usuário

Parâmetros:

Parâmetro	Valores/uso
Tamanho da máscara	3x3 , 5x5 , 7x7 , 9x9
Pesos do kernel	Editáveis quando permitido
Divisor	Usado na convolução

Funcionamento resumido:

```

novo_pixel = soma(pixel_vizinho * peso_da_mascara) / divisor

```

Saída: nova imagem com a mesma largura e altura da entrada.

6.3 Ajuste Ponto a Ponto

Função: aplicar uma operação individual em cada pixel.

Operações disponíveis:

Operação	Regra
Brilho	<code>saida = pixel + valor</code>
Limiarização	<code>pixel >= valor ? 255 : 0</code>

Os valores são limitados ao intervalo `0..255` quando necessário.

Saída: nova imagem com as mesmas dimensões da entrada.

6.4 Visualização

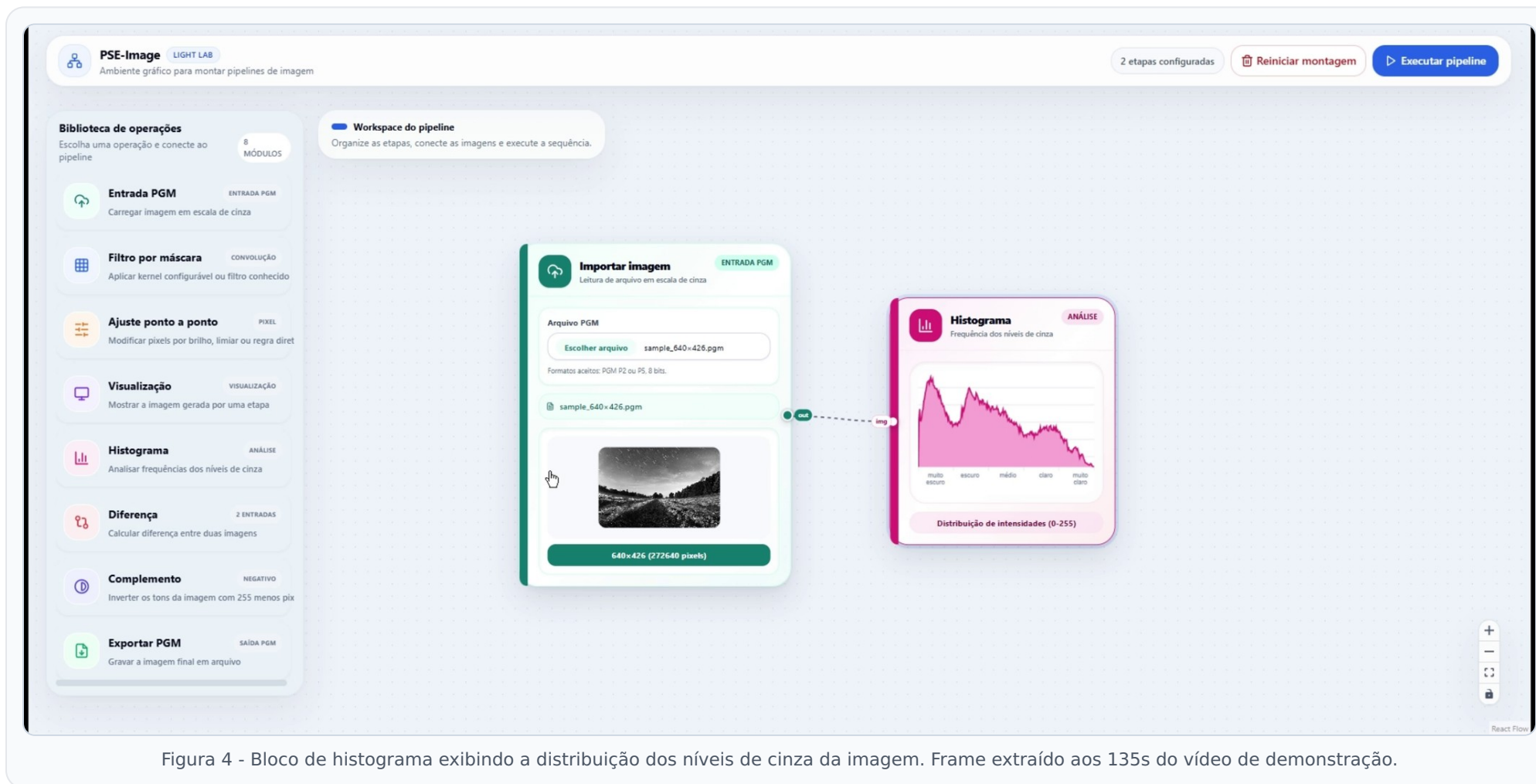
Função: exibir uma imagem no workspace.

Como usar:

1. Conecte uma imagem ao bloco `Visualização`.
2. Execute o pipeline.
3. O resultado aparece em um canvas dentro do bloco.

O bloco também repassa a imagem, permitindo visualizar resultados intermediários sem interromper o fluxo.

6.5 Histograma



Função: calcular e plotar a frequência dos níveis de cinza.

Funcionamento:

```
histograma[pixel] = histograma[pixel] + 1
```

Exemplo:

```
pixels = [0, 0, 128, 255]
histograma[0] = 2
histograma[128] = 1
histograma[255] = 1
```

Saída: gráfico de histograma exibido no frontend.

6.6 Diferença

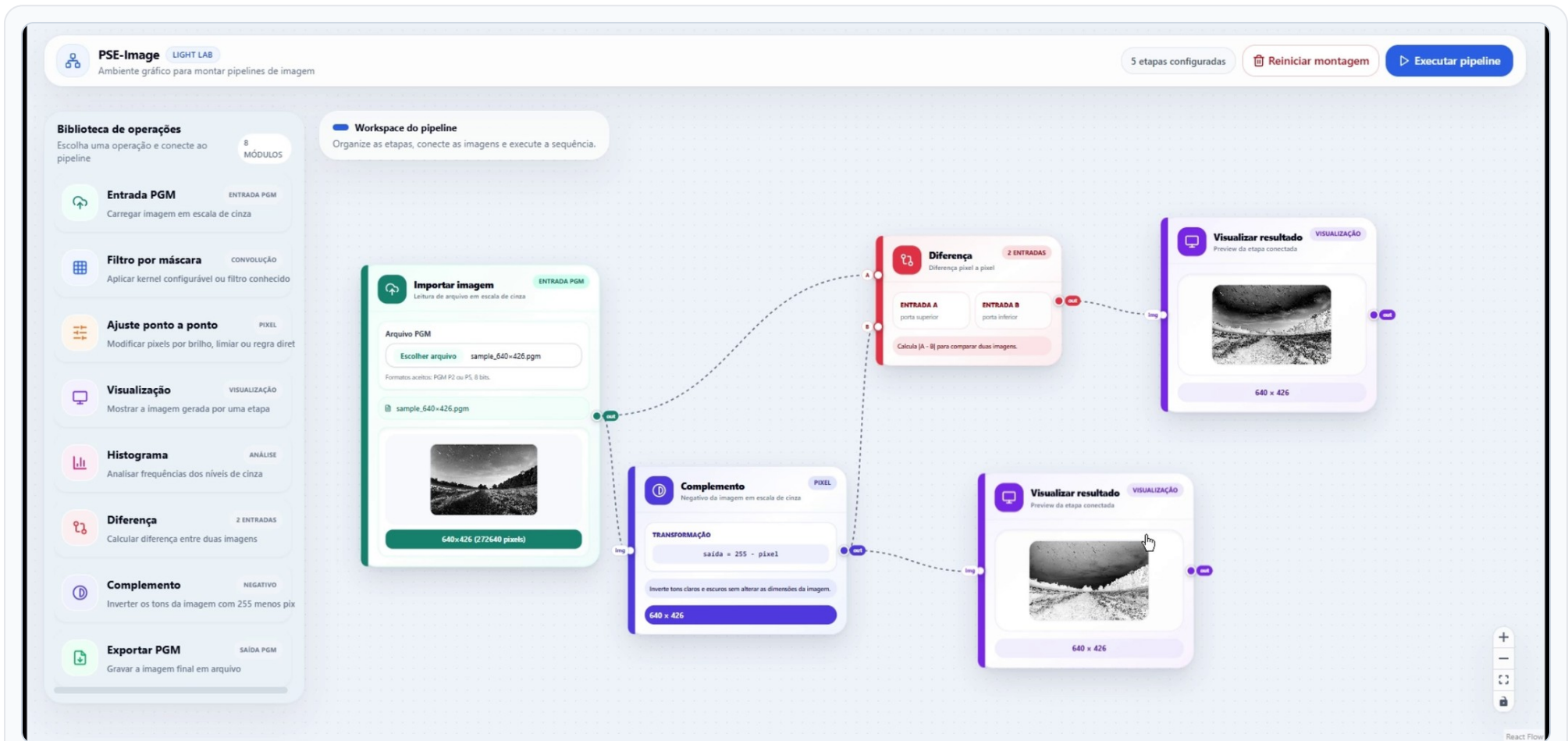


Figura 6 - Comparação entre a imagem original e a processada usando o bloco Diferença. Frame extraído aos 175s do vídeo de demonstração.

Função: comparar duas imagens com as mesmas dimensões.

Regra:

```
saida = abs(pixel_imagem_1 - pixel_imagem_2)
```

Restrição:

Condição	Obrigatória
Mesma largura	Sim
Mesma altura	Sim

Saída: imagem destacando as diferenças entre as entradas.

6.7 Complemento

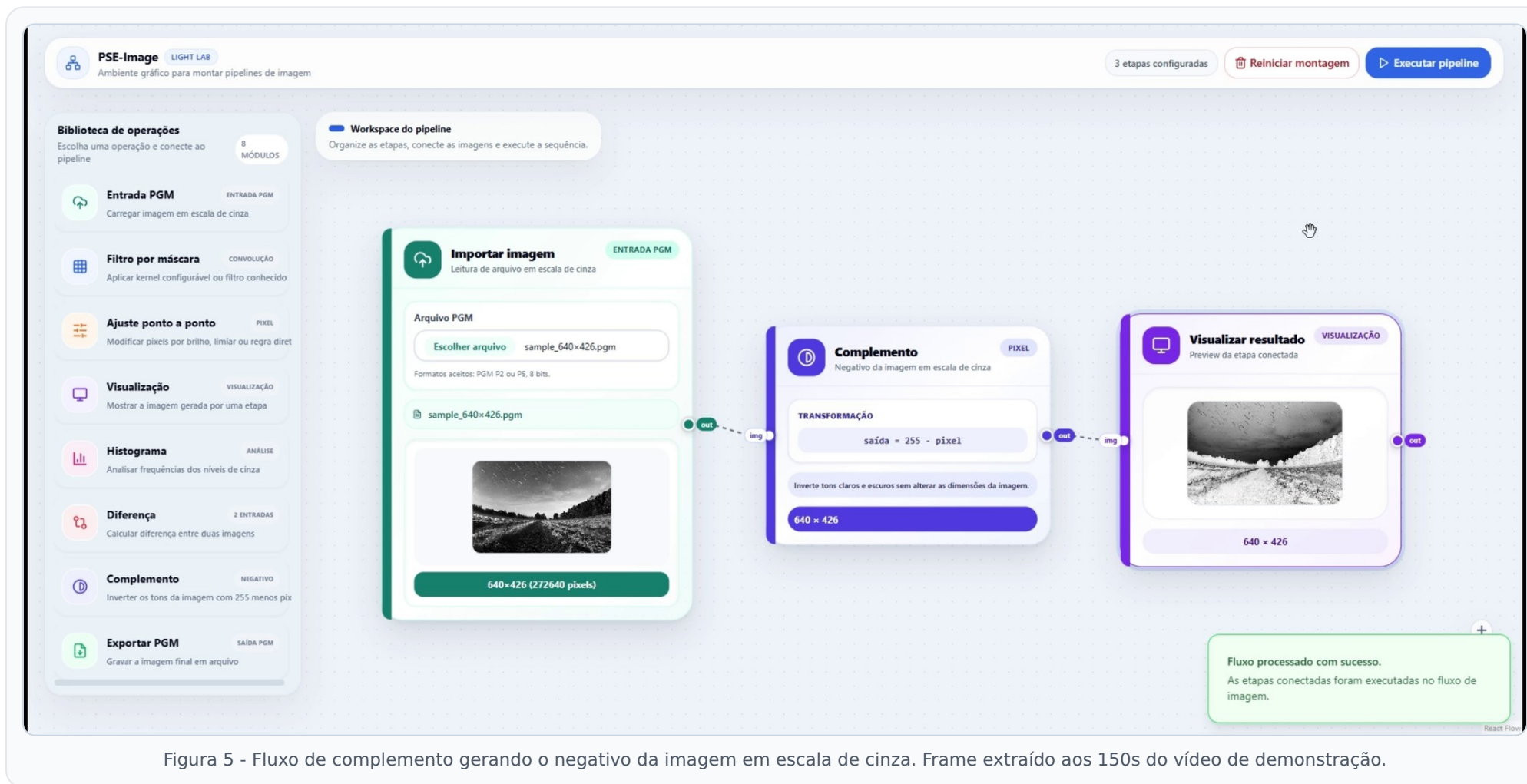


Figura 5 - Fluxo de complemento gerando o negativo da imagem em escala de cinza. Frame extraído aos 150s do vídeo de demonstração.

Função: gerar o negativo da imagem.

Regra:

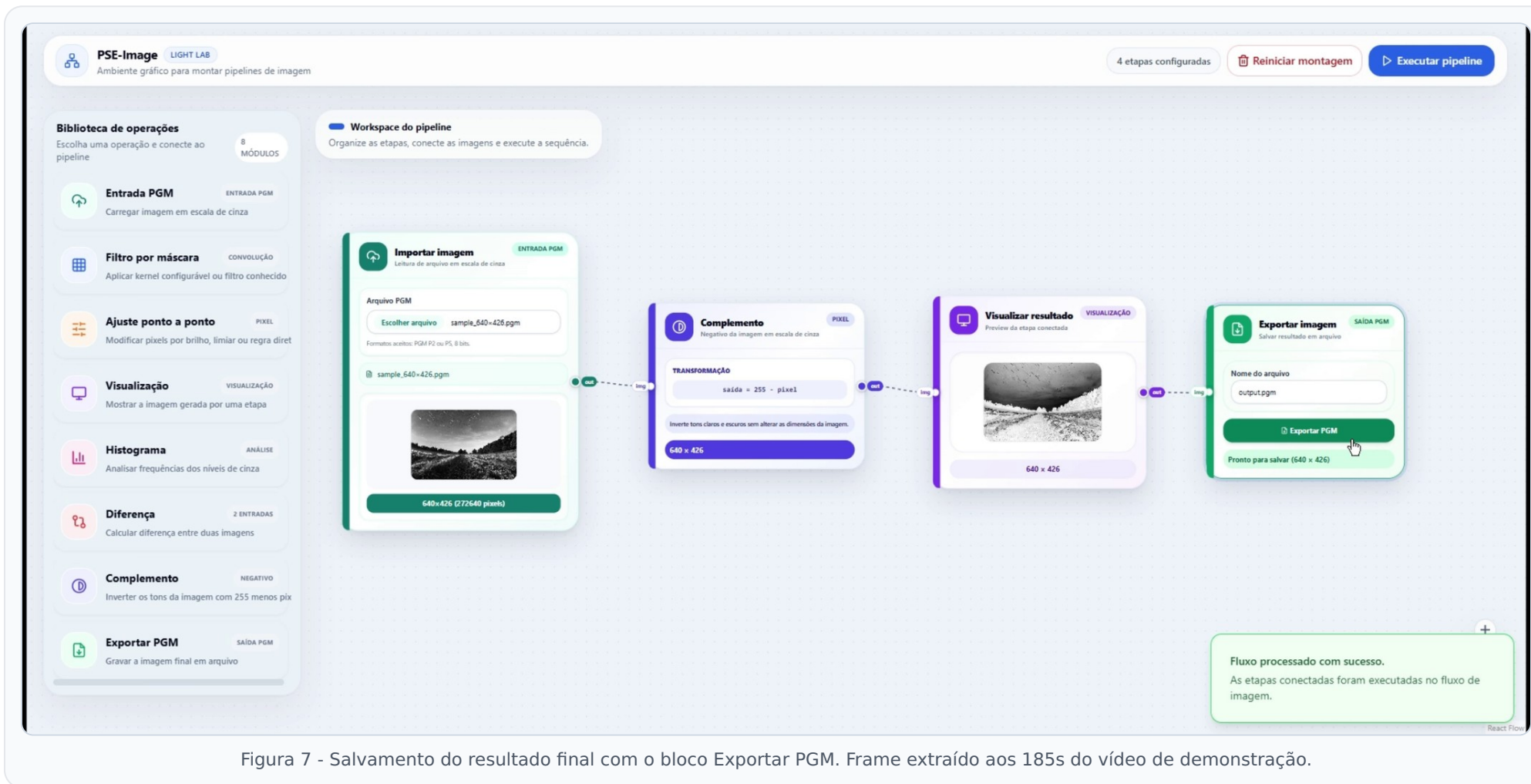
```
saída = 255 - pixel
```

Exemplo:

```
entrada = [0, 10, 128, 255]
saida = [255, 245, 127, 0]
```

Saída: nova imagem com tons invertidos.

6.8 Exportar PGM



Função: salvar uma imagem processada em arquivo PGM.

Como usar:

- 1. Conecte uma imagem ao bloco `Exportar PGM`.
- 2. Informe o nome do arquivo.
- 3. Execute o pipeline.

O backend salva o arquivo em:

```
backend/output/
```

Formato salvo:

Item	Valor
Tipo	PGM binário
Magic number	P5
Valor máximo	255

7. Exemplos de Fluxos

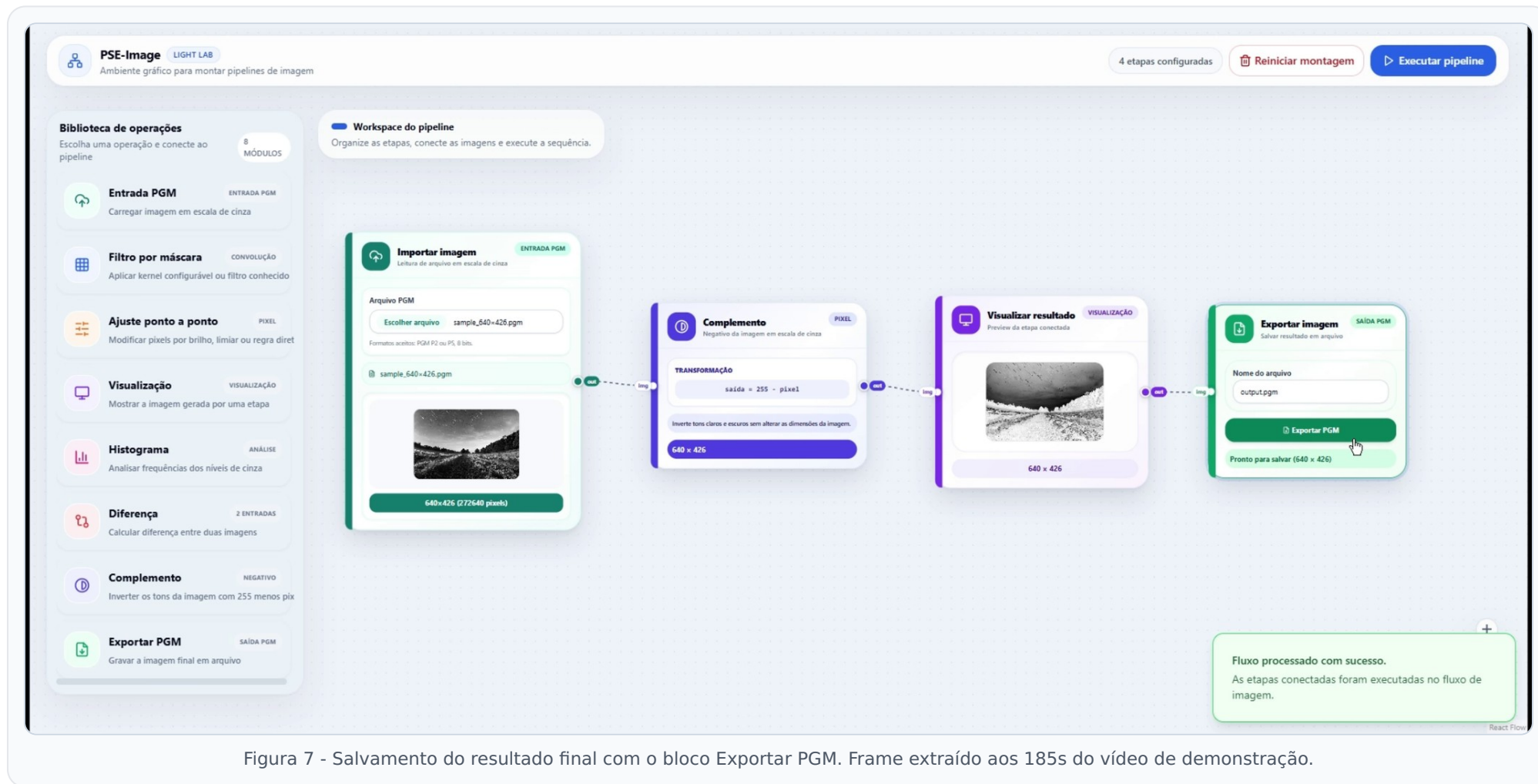


Figura 7 - Salvamento do resultado final com o bloco Exportar PGM. Frame extraído aos 185s do vídeo de demonstração.

7.1 Carregar e visualizar

[Entrada PGM] -> [Visualização]

Uso: verificar se a imagem foi carregada corretamente.

7.2 Aumentar brilho

```
[Entrada PGM] -> [Ajuste ponto a ponto: Brilho] -> [Visualização]
```

Uso: clarear ou escurecer a imagem.

7.3 Limiarização

```
[Entrada PGM] -> [Ajuste ponto a ponto: Limiarização] -> [Visualização]
```

Uso: transformar a imagem em preto e branco usando um valor limite.

7.4 Suavização por média

```
[Entrada PGM] -> [Filtro por máscara: Média] -> [Visualização]
```

Uso: suavizar a imagem.

7.5 Remoção de ruído por mediana

```
[Entrada PGM] -> [Filtro por máscara: Mediana] -> [Visualização]
```

Uso: reduzir ruído preservando bordas.

7.6 Detecção de bordas com laplaciano

```
[Entrada PGM] -> [Filtro por máscara: Laplaciano] -> [Visualização]
```

Uso: destacar regiões de mudança brusca de intensidade.

7.7 Histograma

```
[Entrada PGM] -> [Histograma]
```

Uso: analisar a distribuição dos níveis de cinza.

7.8 Complemento

```
[Entrada PGM] -> [Complemento] -> [Visualização]
```

Uso: gerar o negativo da imagem.

7.9 Comparar original e processada

```
[Entrada PGM] -----> [Diferença] -> [Visualização]
      \                /
      -> [Filtro por máscara: Média] -----
```

Uso: visualizar o quanto uma operação alterou a imagem original.

7.10 Salvar resultado final

```
[Entrada PGM] -> [Complemento] -> [Exportar PGM]
```

Uso: salvar uma imagem processada em `.pgm`.

8. Observações Técnicas

- O processamento das imagens é feito manualmente em Python.
 - O backend não usa métodos prontos de bibliotecas de visão computacional.
 - O grafo é executado em ordem topológica.
 - Cada resultado fica associado ao id do nó correspondente.
 - Os pixels são representados como inteiros entre 0 e 255.
 - Operações que geram imagem preservam largura e altura da entrada.
 - O bloco histograma gera dados de análise, não uma imagem.
 - O bloco salvar gera um arquivo PGM e retorna metadados do salvamento.
-

9. Encerramento

Para encerrar a aplicação:

1. No terminal do frontend, pressione `Ctrl + C`.
2. No terminal do backend, pressione `Ctrl + C`.
3. Se estiver usando ambiente virtual Python, execute:

```
deactivate
```
